

Nav2 플래너

글로벌 플래너 (NavFn, Smac) & DWB 로컬 플래너

cchyun@gmail.com

Part 1. 글로벌 경로 계획

글로벌 경로 계획(Global Path Planning)이란?

- 로봇이 이미 알고 있는 전체 지도를 바탕으로, 출발지에서 목적지까지 가는 가장 효율적인 경로를 미리 계산하는 과정
 - 스마트폰 내비게이션과 같은 역할
 - 목적지까지의 전체 경로를 미리 보여주는 것과 동일
 - 로봇은 이를 통해 '어떤 복도를 지나 어느 문으로 들어갈지' 거시적 전략을 수립
 - 글로벌 플래너만으로 자율주행이 안 되는 이유
 - 글로벌 플래너는 보통 정적인(변하지 않는) 지도를 기준으로 계획
 - 주행 중 갑자기 나타난 사람, 이동식 장애물 등 동적인 상황에는 실시간 대응 어려움
 - 실제 주행 시에는 눈앞의 장애물을 피하는 로컬 플래너(컨트롤러)의 도움이 반드시 필요

Part 2. NavFn 플래너

- NavFn(Navigation Function): ROS 시절부터 사용되어 온 전통적이고 안정적인 플래너
 - Dijkstra 알고리즘 기반
 - 전체 지도에서 출발지로부터 모든 방향으로 비용(Cost)을 계산하며 뺏어나감
 - 목적지까지의 **가장 낮은 비용(최단 거리)** 을 갖는 경로 탐색
 - 물을 뿌렸을 때 가장 낮은 곳으로 흘러가는 경로를 찾는 것과 유사
 - use_astar 옵션
 - 파라미터 설정을 통해 **A*(에이 스타) 알고리즘**으로 전환 가능
 - A*는 단순히 모든 방향을 탐색하는 대신 '목적지 방향'이라는 힌트(휴리스틱)를 사용하여 더 빠르게 경로 탐색

- NavFn의 장단점

- 장점

- 계산 속도가 매우 빠르고 구현이 간단하여 안정적

- 단점

- 로봇을 하나의 '점(Particle)'으로 가정하기 때문에 로봇의 회전 반경이나 크기 같은 **운동학적 제약(비홀로노믹 특성)** 무시
- 로봇이 실제로 따라가기 힘든 급격한 꺾임이 있는 경로를 생성하기도 함

- TurtleBot3에 적합한 이유

- TurtleBot3는 차동 구동 방식으로 제자리 회전이 가능
- NavFn은 **홀로노믹 가정**(로봇을 점으로 취급, 운동학적 제약 무시)을 사용
- 작은 크기와 단순한 실내 환경에서는 복잡한 운동학적 제약이 **덜 중요**

```
planner_server:
  ros__parameters:
    use_sim_time: True
    expected_planner_frequency: 20.0
    planner_plugins: ["GridBased"]
    GridBased:
      plugin: "nav2_navfn_planner/NavfnPlanner"
      tolerance: 0.5
      use_astar: false
      allow_unknown: true
```

Part 3. Smac 플래너

- Smac Planner: 로봇의 물리적인 움직임 한계를 고려하여 실제로 주행 가능한(Kinematically Feasible) 경로를 생성
 - Hybrid-A* 알고리즘
 - 전통적인 격자 기반 탐색에 로봇의 실제 주행 모델을 결합
 - 자동차처럼 회전 반경이 필요한 모델도 처리 가능
 - NavFn 대비 장점
 - 로봇의 최소 회전 반경과 방향(Orientation) 을 계산에 포함
 - 로봇이 벽에 걸리거나 불가능한 각도로 회전해야 하는 상황을 사전에 방지
 - 부드러운 곡선 경로 생성

- SmacPlannerHybrid vs SmacPlannerLattice

- SmacPlannerHybrid

- 자동차나 모바일 로봇처럼 전진/후진과 회전 반경의 제약이 뚜렷한 로봇에 최적화
- SE2(X, Y, 방향) 공간에서 탐색하며, 후진 경로도 생성 가능

- SmacPlannerLattice

- 로봇의 가능한 움직임들을 미리 계산된 세트(Lattice)로 만들어 탐색
- 로봇의 모양이 원형이 아니거나 아주 복잡한 움직임이 필요한 경우 유연하게 대처 가능
- 원형이 아닌 특수한 형태의 로봇에 적합

글로벌 플래너 선택 기준

상황	추천 플래너	이유
단순한 실내 환경	NavFn	계산량이 적어 저사양 로봇에서도 매우 빠르게 작동하며 안정적
좁은 통로나 복잡한 커브	Smac 2D	비용 인식(Cost-aware) 기능으로 장애물에서 멀어지는 경로 생성
자동차형(Ackermann) 로봇	Smac (Hybrid)	제자리 회전이 안 되는 로봇 특성 반영
특수 형태 로봇	Smac (Lattice)	로봇 고유의 복잡한 움직임을 미리 정의된 제어 세트로 처리

Part 4. 글로벌 경로 시각화 및 실패 원인

- 로봇이 계산한 글로벌 경로는 ROS 2 토픽을 통해 확인 가능
 - 토픽 정보
 - 토픽 이름: `/plan`
 - 메시지 타입: `nav_msgs/msg/Path`
 - RViz2 확인 방법
 - RViz2 실행 후 왼쪽 하단의 **[Add]** 버튼 클릭
 - **[By topic]** 탭에서 `/plan` 토픽 아래의 **Path**를 선택하여 추가
 - 지도 위에 생성된 선 형태의 경로를 통해 로봇이 계획한 길을 실시간으로 모니터링

- 경로 생성 실패 시 주요 원인 4가지

- ① 목적지가 장애물 안(Lethal 영역)에 있음

- 목적지가 벽이나 장애물과 너무 가깝거나 겹쳐 있을 때 발생
- 해결: 목적지를 장애물에서 조금 떨어진 자유 공간(Free space)으로 재지정

- ② 연결된 경로가 없음

- 문이 닫혀 있거나 지도상의 틈이 로봇 크기보다 좁은 경우
- 해결: 로봇의 크기 파라미터(Footprint)를 조정하거나 지도를 수정

- 경로 생성 실패 시 주요 원인 (계속)

- ③ Costmap 미업데이트

- 센서 연결 문제로 지도가 최신화되지 않아 실제로는 뚫려 있는 길이 막힌 것으로 표시
- 해결: 센서 데이터 토픽과 TF(좌표 변환) 상태가 정상인지 확인

- ④ 최대 탐색 횟수 초과

- 지도가 너무 넓어 경로를 찾는 데 시간이 너무 오래 걸리는 경우
- 해결: `max_iterations` 파라미터 값을 높여 탐색 기회를 추가 부여

- ⑤ `allow_unknown` 설정 오류

- `allow_unknown: false` 이면 탐색되지 않은 영역을 장애물처럼 취급
- 해결: 미탐색 영역 진입이 필요하다면 `allow_unknown: true` 로 설정

Part 5. DWB 로컬 플래너

- 두 플래너의 관계: 내비게이션(GPS)과 운전자의 눈·핸들 조작
 - 글로벌 플래너 (큰 그림)
 - 자동차의 **네비게이션(GPS)** 과 같은 역할
 - 전체 지도를 보고 어떤 복도와 방을 지나갈지 커다란 경로를 설계
 - 고정된 벽이나 기둥 등 **정적인 장애물**을 피해 효율적인 전체 루트 생성
 - 로컬 플래너 (순간 대응)
 - 운전자의 눈과 핸들 조작과 같은 역할
 - 글로벌 플래너가 준 경로를 따라가면서 갑자기 나타난 사람이나 물체 등 **동적인 장애물**을 실시간으로 감지하고 회피
 - 순간순간 로봇의 바퀴에 얼마만큼의 속도를 줄지 결정하는 "**순발력**" 담당

- DWA: 로봇의 물리적 한계 내에서 최적의 속도를 찾는 알고리즘

1. Dynamic Window (속도 공간)

- 현재 속도에서 가속도 한계(`acc_lim`) 내에서 실제로 도달할 수 있는 **실현 가능한 속도 범위** 정의

2. 속도 샘플링

- 실현 가능한 범위 안에서 여러 개의 선속도(v)와 각속도(ω) 조합을 선택

3. 예측 경로 시뮬레이션

- 선택된 각 속도 조합으로 로봇이 `sim_time` 동안 움직였을 때 예상되는 궤적(주로 원호 형태)을 시뮬레이션

4. 최적 경로 선택

- 장애물 거리, 글로벌 경로 일치성 등을 평가하여 **가장 점수가 높은 속도**를 선택해 모터 명령 전송

- DWB: 경로 평가 기준인 'Critic'을 플러그인 형태로 자유롭게 교체 가능하도록 모듈화
 - GoalAlign
 - 로봇의 방향이 최종 목적지 방향과 얼마나 잘 일치하는지 평가
 - PathAlign
 - 로봇이 글로벌 플래너가 생성한 경로의 방향을 얼마나 잘 따라가고 있는지 평가
 - GoalDist
 - 예측 경로의 끝점이 최종 목적지와 얼마나 가까운지 평가하여 목적지로 향하게 유도
 - PathDist
 - 로봇이 글로벌 경로에서 물리적으로 얼마나 떨어져 있는지 평가하여 경로 이탈 방지
 - Oscillation
 - 로봇이 제자리에서 앞뒤로 왔다 갔다 하며 진동하는 현상을 방지

DWB 핵심 파라미터 및 성능 영향

파라미터	설명	성능 영향
min_vel_x / max_vel_x	최소/최대 선속도	최대 속도가 높으면 빨리 도착하지만 제어 어려움
min_vel_theta / max_vel_theta	최소/최대 각속도	너무 낮으면 좁은 길에서 회전이 답답할 수 있음
acc_lim_x / acc_lim_theta	선가속도 및 각가속도 한계	이 값이 크면 기민하게 반응하지만 주행이 거칠어짐
vx_samples / vtheta_samples	속도 공간에서 추출할 샘플 수	많을수록 정밀한 제어 가능하지만 CPU 부하 증가
sim_time	경로 예측 시뮬레이션 시간	길수록 먼 미래를 예측하지만 계산량이 늘고 장애물 대응이 늦어짐

- 로컬 플래너가 경로를 생성할 수 없거나 글로벌 경로에서 너무 멀어지면 Behavior Tree(BT)가 감지하여 recovery_server 실행

1. Costmap 초기화

- 감지된 장애물 정보가 잘못되었을 수 있으므로 지도를 깨끗이 초기화

2. 제자리 회전 (Spin)

- 주변 공간을 다시 확인하기 위해 제자리에서 회전

3. 후진 (Backup)

- 좁은 길에 갇혔을 경우 뒤로 물러나 새로운 공간 탐색

4. 모든 시도 실패 시

- 내비게이션 작업 전체를 중단하고 에러 보고

- 주요 DWB 실패 원인 4가지

- 속도 범위 설정 오류

- 로봇의 물리적 성능보다 높은 속도/가속도를 설정하면 주행 중 경로 이탈 또는 미끄러짐
- 해결: 실제 모터 사양에 맞춰 파라미터 수정

- 진동 현상 (Oscillation)

- `Oscillation` Critic의 가중치를 높이거나, 가속도 한계를 낮추어 급격한 방향 전환 억제

- 목적지 직전 멈춤/회전

- `xy_goal_tolerance` 나 `yaw_goal_tolerance`가 너무 낮으면 목적지 주변에서 계속 맴돌거나 멈춤
- 해결: 허용 오차를 적절히 높여 설정

- 좁은 길 통과 실패

- 로봇의 반지름(`robot_radius`)이나 Footprint 설정이 실제보다 크면 통과 가능한 길도 장애물로 인식
- 해결: 정확한 크기 설정 필요

기본 DWB 파라미터 YAML 예시 (1/3)

```
controller_server:
  ros__parameters:
    use_sim_time: True
    controller_frequency: 20.0          # 컨트롤러 실행 주기 (Hz). 20Hz = 0.05초마다 속도 명령 계산
    min_x_velocity_threshold: 0.001     # X방향 최소 속도 임계값 (m/s). 이 값 미만은 0으로 처리 (노이즈 제거)
    min_y_velocity_threshold: 0.5       # Y방향 최소 속도 임계값 (m/s). 차동구동 로봇은 Y방향 이동 불가라 높게 설정
    min_theta_velocity_threshold: 0.001 # 각속도 최소 임계값 (rad/s). 미세 회전 노이즈 제거
    failure_tolerance: 0.3              # 컨트롤러 실패 허용 시간 (초). 0.3초간 이상 있으면 recovery 동작
    progress_checker_plugin: "progress_checker" # 진행 상황 체크에 사용할 플러그인 이름
    goal_checker_plugins: ["general_goal_checker"] # 목표 도달 판정에 사용할 플러그인 목록
    controller_plugins: ["FollowPath"] # 로컬 경로 추종에 사용할 플러그인 목록 (여기서는 DWB 사용)

progress_checker:
  plugin: "nav2_controller::SimpleProgressChecker" # 로봇이 실제로 움직이고 있는지 감시하는 플러그인
  required_movement_radius: 0.5                 # 진행으로 인정할 최소 이동 반경 (m). 0.5m 이상 이동해야 진행 중으로 판단
  movement_time_allowance: 10.0                 # 이동을 기다릴 최대 시간 (초). 10초 동안 0.5m 이동 없으면 정지로 간주

general_goal_checker:
  stateful: True                                # 목표 도달 상태를 유지 (한 번 도달하면 계속 도달로 간주)
  plugin: "nav2_controller::SimpleGoalChecker" # 단순 위치+자세 기반 목표 도달 검사 플러그인
  xy_goal_tolerance: 0.25                       # 위치 목표 허용 오차 (m). 목표 지점 반경 25cm 이내 도달 시 성공
  yaw_goal_tolerance: 0.25                      # 자세(방향) 목표 허용 오차 (rad). 약 14.3도(0.25rad) 이내 정렬 시 성공
```

기본 DWB 파라미터 YAML 예시 (2/3)

```
FollowPath:
  plugin: "dwb_core::DWBLocalPlanner" # DWB (Dynamic Window Approach) 로컬 플래너 플러그인
  debug_trajectory_details: True      # 디버깅용 상세 궤적 정보 출력 (개발 시 유용, 운용 시는 False 권장)
  min_vel_x: 0.0                     # X방향 최소 속도 (m/s). 0이면 후진 불가 (전진만)
  min_vel_y: 0.0                     # Y방향 최소 속도 (m/s). 차동구동은 0 (옴니휠은 음수 가능)
  max_vel_x: 0.26                     # X방향 최대 속도 (m/s). 로봇의 전진 최고 속도 제한
  max_vel_y: 0.0                     # Y방향 최대 속도 (m/s). 차동구동은 0 (횡방향 이동 불가)
  max_vel_theta: 1.0                 # 최대 각속도 (rad/s). 로봇의 최대 회전 속도 (약 57도/초)
  min_speed_xy: 0.0                  # XY 평면 최소 속도 (m/s). 0이면 완전 정지 가능
  max_speed_xy: 0.26                 # XY 평면 최대 속도 (m/s). max_vel_x와 동일하게 제한
  min_speed_theta: 0.0               # 최소 각속도 (rad/s). 0이면 회전 정지 가능
  acc_lim_x: 2.5                     # X방향 최대 가속도 (m/s2). 0.26m/s까지 가속하는 데 필요한 가속력
  acc_lim_y: 0.0                     # Y방향 최대 가속도 (m/s2). 차동구동은 0
  acc_lim_theta: 3.2                 # 최대 각가속도 (rad/s2). 회전 가속도 제한 (급격한 회전 방지)
  decel_lim_x: -2.5                  # X방향 최대 감속도 (m/s2). 음수 = 감속. 급제동 방지 및 안정적 정지
  decel_lim_y: 0.0                   # Y방향 최대 감속도 (m/s2). 차동구동은 0
  decel_lim_theta: -3.2              # 최대 각감속도 (rad/s2). 회전 감속도 제한
  vx_samples: 20                     # X방향 속도 샘플 개수. 0~0.26을 20개로 분할해 후보 속도 탐색
  vy_samples: 5                      # Y방향 속도 샘플 개수. 차동구동은 의미 없지만 옴니휠용으로 남김
  vtheta_samples: 20                 # 각속도 샘플 개수. 회전 속도 후보를 20개로 분할 탐색
  sim_time: 1.7                      # 궤적 시뮬레이션 시간 (초). 1.7초 미래까지 예측해 장애물 회피 및 경로 평가
  linear_granularity: 0.05           # 선형 궤적 해상도 (m). 궤적을 5cm 간격으로 샘플링
  angular_granularity: 0.025         # 각도 궤적 해상도 (rad). 궤적의 각도 변화를 0.025rad 간격으로 샘플링
  transform_tolerance: 0.2           # TF 변환 허용 시간 (초). 0.2초 이내의 좌표 변환만 신뢰
  xy_goal_tolerance: 0.25            # 컨트롤러 낮은 수준의 위치 허용 오차 (m). goal_checker와 별개로 동작
  trans_stopped_velocity: 0.25       # 정지 판정 기준 속도 (m/s). 이 값 이하면 "멈춤"으로 간주
  short_circuit_trajectory_evaluation: True # 평가 중 불량 궤적은 즉시 폐기하여 계산 속도 향상
  stateful: True                     # 낮은 수준에서도 상태를 유지 (goal_checker의 stateful과 유사)
```




```
critics: ["RotateToGoal", "Oscillation", "BaseObstacle", "GoalAlign", "PathAlign", "PathDist", "GoalDist"]
# DWB는 여러 크리티크(평가 함수)으로 후보 궤적을 점수 매겨 최적의 속도 명령 선택
# RotateToGoal: 목표 근처에서 정확한 방향을 향하도록 유도
# Oscillation: 같은 위치를 왔다갔다하는 진동(oscillation)을 감지하고 패널티 부여
# BaseObstacle: 장애물과의 거리를 평가. 가까울수록 높은 비용
# GoalAlign: 궤적이 목표 방향과 일치하도록 정렬 유도
# PathAlign: 궤적이 전역 경로와 정렬되도록 유도
# PathDist: 후보 궤적이 전역 경로에서 얼마나 떨어져 있는지 평가
# GoalDist: 후보 궤적이 목표 지점에 얼마나 가까워지는지 평가
BaseObstacle.scale: 0.02 # 장애물 회피 가중치. 낮은 값 = 장애물 회피보다 경로 추종 우선
PathAlign.scale: 32.0 # 경로 정렬 가중치. 높은 값 = 전역 경로를 철저히 따라가도록 유도
PathAlign.forward_point_distance: 0.1 # 경로 정렬 시 참조할 전방 지점 거리 (m). 10cm 앞 경로 방향 참조
GoalAlign.scale: 24.0 # 목표 정렬 가중치. 높은 값 = 목표 방향 정렬 강하게 유도
GoalAlign.forward_point_distance: 0.1 # 목표 정렬 시 참조할 전방 지점 거리 (m)
PathDist.scale: 32.0 # 경로 거리 가중치. 높은 값 = 전역 경로로부터 벗어나지 않도록 강하게 유도
GoalDist.scale: 24.0 # 목표 거리 가중치. 높은 값 = 목표 지점에 최대한 가깝게 유도
RotateToGoal.scale: 32.0 # 목표 회전 가중치. 높은 값 = 목표 방향 정확히 맞추도록 강하게 유도
RotateToGoal.slowing_factor: 5.0 # 목표 근처 감속 계수. 값이 클수록 목표에 가까울수록 천천히 회전
RotateToGoal.lookahead_time: -1.0 # 목표 회전 시 미리보기 시간 (초). -1 = 자동 계산 (sim_time 사용)
```